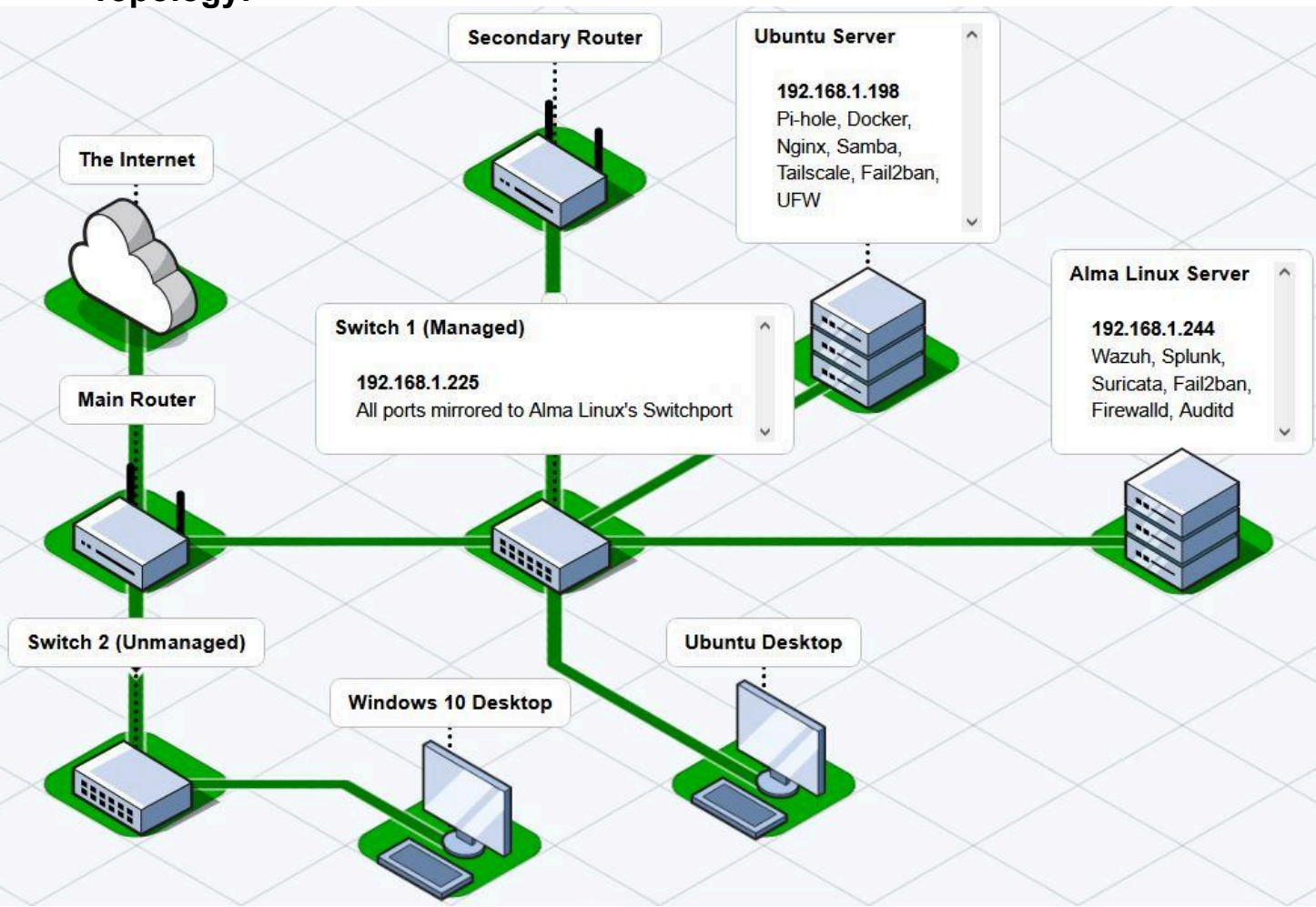
Eric Guzman 2/25/2026

SOC Analyst Homelab: Suricata, Wazuh, and Splunk Threat Detection Workflow

Executive Summary: This report documents a network security monitoring environment built on a single host running AlmaLinux 10.1, with Suricata providing network layer detection, Wazuh providing host layer correlation, and Splunk Enterprise serving as the central SIEM. The environment captured and analyzed 13,618 events across a monitored window, including Priority 1 reconnaissance detection, automated threat intelligence matches from Spamhaus and ET CINS feeds, and a confirmed SSH brute force activity detected through Wazuh's rule correlation engine, with rule 5712 firing 39 times against authentic inbound bot traffic. Log analysis was performed at both the CLI and SIEM layer, following a triage workflow that mirrors standard SOC practice: confirm ingestion, categorize by severity, isolate high-priority events, and build a timeline. This report documents that workflow end to end.

Topology:



Tools and Stack:

Suricata v7.0.8 (built from source) Network IDS: signature-based packet inspection, EVE JSON logging	**Wazuh** v4.14.3 Host IDS + SIEM agent: log analysis, rule correlation, alerting	**Splunk Enterprise** v10.2 SIEM: log ingestion, SPL querying, dashboards, event correlation
AlmaLinux v10.1 Base OS: RHEL-compatible enterprise Linux environment	**Nmap** v7.95 Reconnaissance tool: used to generate network scan traffic and test IDS rules	**OpenSSH** v9.9 SSH service: authentication monitoring and brute force detection testing

Skills Demonstrated:

Linux administration: Package management, service config, systemctl on AlmaLinux/RHEL	**Log analysis:** CLI filtering (grep, tail), pattern recognition, noise vs. signal
Suricata IDS: Source build, rule management, EVE JSON log analysis, fast.log triage	**SIEM investigation:** Time-scoping, source correlation, alert triage workflow
Wazuh HIDS: Agent/manager deployment, rule escalation chains, alert investigation	**Threat detection:** Signature-based (Suricata) + behavior-based (Wazuh) detection methods
Splunk Enterprise: Data ingestion, forwarder config, SPL queries, stats and timechart	**Security documentation:** Incident timeline, IOC tables, findings write-up, analyst reporting

Detection Results:

Suricata **Network layer**	13,618 events captured and analyzed across the monitoring window. Suricata detected SSH protocol anomalies, fired on two live threat intelligence feeds (ET CINS, ET COMPROMISED), and produced structured EVE JSON logs ingested directly into Splunk.
Wazuh **Host layer**	SSH authentication failures from a publicly exposed server triggered Wazuh rule 5712 (level 10) 39 times against real inbound bot traffic. The frequency correlation engine confirmed brute force activity automatically across multiple external source IPs.

Splunk SIEM layer	Wrote SPL queries to ingest, filter, and analyze Suricata EVE JSON. Queries covered: event counts by source IP, stats count by alert.severity, top targeted ports, time-scoped incident windows, and cross-source correlation with Wazuh data. Demonstrates practical SIEM proficiency.
Log Analysis CLI + SIEM	Raw logs filtered at the CLI using grep and tail -f before pivoting into Splunk for structured analysis. This mirrors the real analyst workflow: triage at the command line, investigate in the SIEM, document the findings.

Section 1: Suricata IDS Log Analysis -

Suricata monitors all traffic that passes through the switch as SPAN is configured on all ports and sends all traffic directly to Suricata where it compares it against the Emerging Threats ruleset. When traffic matches a rule Suricata writes an alert to fast.log and a JSON record to eve.json. This JSON output is what Splunk ingests while the fast.log is seen in a terminal for a quick triage.

1a. Live Background Traffic

To view alerts in real time I ran:

$ sudo tail -20 /var/log/suricata/fast.log

```
[eg@eg-HP-Pavilion-x360-Convertible-14m-cd0xxx ~]$ sudo tail -20 /var/log/suricata/fast.log
02/27/2026-18:32:43.678526  [**] [1:2260002:1] SURICATA Applayer Detect protocol only one direction [**] [Classification: Generic Protocol Command Decode] [Priority: 3]
 {TCP} 192.168.1.198:2222 -> 180.106.83.59:48929
02/27/2026-18:32:43.678526  [**] [1:2228000:1] SURICATA SSH invalid banner [**] [Classification: Generic Protocol Command Decode] [Priority: 3] {TCP} 192.168.1.198:2222
 -> 180.106.83.59:48929
02/27/2026-18:32:43.906579  [**] [1:2228000:1] SURICATA SSH invalid banner [**] [Classification: Generic Protocol Command Decode] [Priority: 3] {TCP} 180.106.83.59:4892
9 -> 192.168.1.198:2222
02/27/2026-18:32:54.559623  [**] [1:2210044:2] SURICATA STREAM Packet with invalid timestamp [**] [Classification: Generic Protocol Command Decode] [Priority: 3] {TCP}
192.168.1.244:55298 -> 206.82.17.213:80
02/27/2026-18:32:54.567610  [**] [1:2210044:2] SURICATA STREAM Packet with invalid timestamp [**] [Classification: Generic Protocol Command Decode] [Priority: 3] {TCP}
192.168.1.244:55298 -> 206.82.17.213:80
02/27/2026-18:32:54.567611  [**] [1:2210044:2] SURICATA STREAM Packet with invalid timestamp [**] [Classification: Generic Protocol Command Decode] [Priority: 3] {TCP}
192.168.1.244:55298 -> 206.82.17.213:80
02/27/2026-18:33:14.414004  [**] [1:2260002:1] SURICATA Applayer Detect protocol only one direction [**] [Classification: Generic Protocol Command Decode] [Priority: 3]
 {TCP} 192.168.1.198:2222 -> 14.103.115.124:38296
02/27/2026-18:33:14.414004  [**] [1:2228000:1] SURICATA SSH invalid banner [**] [Classification: Generic Protocol Command Decode] [Priority: 3] {TCP} 192.168.1.198:2222
 -> 14.103.115.124:38296
02/27/2026-18:33:14.654848  [**] [1:2228000:1] SURICATA SSH invalid banner [**] [Classification: Generic Protocol Command Decode] [Priority: 3] {TCP} 14.103.115.124:382
96 -> 192.168.1.198:2222
02/27/2026-18:33:49.601681  [**] [1:2228000:1] SURICATA SSH invalid banner [**] [Classification: Generic Protocol Command Decode] [Priority: 3] {TCP} 14.103.115.124:378
32 -> 192.168.1.198:2222
02/27/2026-18:33:49.601681  [**] [1:2228000:1] SURICATA SSH invalid banner [**] [Classification: Generic Protocol Command Decode] [Priority: 3] {TCP} 192.168.1.198:2222
 -> 14.103.115.124:37832
02/27/2026-18:33:58.323673  [**] [1:2027397:1] ET INFO Spotify P2P Client [**] [Classification: Not Suspicious Traffic] [Priority: 3] {UDP} 192.168.1.164:57621 -> 192.1
68.1.255:57621
02/27/2026-18:34:21.025827  [**] [1:2260002:1] SURICATA Applayer Detect protocol only one direction [**] [Classification: Generic Protocol Command Decode] [Priority: 3]
 {TCP} 192.168.1.198:2222 -> 14.103.115.124:37292
02/27/2026-18:34:21.025827  [**] [1:2228000:1] SURICATA SSH invalid banner [**] [Classification: Generic Protocol Command Decode] [Priority: 3] {TCP} 192.168.1.198:2222
 -> 14.103.115.124:37292
02/27/2026-18:34:21.265493  [**] [1:2228000:1] SURICATA SSH invalid banner [**] [Classification: Generic Protocol Command Decode] [Priority: 3] {TCP} 14.103.115.124:372
92 -> 192.168.1.198:2222
02/27/2026-18:34:28.555434  [**] [1:2027695:5] ET INFO Observed Cloudflare DNS over HTTPS Domain (cloudflare-dns .com in TLS SNI) [**] [Classification: Misc activity] [
Priority: 3] {TCP} 192.168.1.198:57818 -> 1.1.1.1:443
02/27/2026-18:34:53.348804  [**] [1:2228000:1] SURICATA SSH invalid banner [**] [Classification: Generic Protocol Command Decode] [Priority: 3] {TCP} 14.103.115.124:354
38 -> 192.168.1.198:2222
02/27/2026-18:34:53.348804  [**] [1:2228000:1] SURICATA SSH invalid banner [**] [Classification: Generic Protocol Command Decode] [Priority: 3] {TCP} 192.168.1.198:2222
 -> 14.103.115.124:35438
02/27/2026-18:36:17.995250  [**] [1:2403375:106951] ET CINS Active Threat Intelligence Poor Reputation IP group 76 [**] [Classification: Misc Attack] [Priority: 2] {TCP
} 47.253.170.224:53906 -> 192.168.1.198:2222
02/27/2026-18:38:58.400080  [**] [1:2027397:1] ET INFO Spotify P2P Client [**] [Classification: Not Suspicious Traffic] [Priority: 3] {UDP} 192.168.1.164:57621 -> 192.1
68.1.255:57621
[eg@eg-HP-Pavilion-x360-Convertible-14m-cd0xxx ~]$
```

Running sudo tail -20 -f /var/log/suricata/fast.log shows alerts firing continuously, not just during the scan. This is normal as Suricata monitors everything on the interface. The background traffic visible here includes Spotify P2P, Cloudflare DNS, inbound Spamhaus-listed IPs hitting the SSH port, and SSH-2.0-Go connections from the PiHole service. These fire whether or not a scan is in progress, but it is my job to cut through the noise and look for what is background traffic and what is suspicious.

SIGNATURE	WHAT IT MEANS
ET DROP Spamhaus / ET CINS Poor Reputation IP Priority 2 - Misc Attack	Inbound connection attempts from IPs on the Spamhaus and CINS threat intelligence blocklists, hitting the SSH port. Traffic direction is toward the host, automated bots scanning the internet for open SSH ports. Not related to the nmap scan; this is normal internet background noise on any exposed host.
ET COMPROMISED Known Compromised or Hostile Host Priority 2 Misc Attack	Source IP matched the Spamhaus compromised host list, a separate feed from CINS. Another inbound bot. Two independent threat intel feeds flagging the same category of unsolicited inbound activity.
ET INFO SSH-2.0-Go version string Priority 3 Misc Activity	A Go-language SSH client on the network making outbound connections. This is the PiHole service, not malicious. Suricata logs it because the client version string is identifiable, a good example of why context matters when reading alerts.
ET INFO Spotify P2P Client Priority 3 Not Suspicious	Spotify peer-to-peer traffic. Fires because the ET ruleset logs known P2P protocols. Priority 3, Not Suspicious, Suricata is doing its job by logging everything; the analyst's job is deciding what matters.

1b. Nmap scan Traffic

To show how active scan traffic looks I ran the same tail command while also performing a nmap scan:

$ sudo tail -f /var/log/suricata/fast.log

```
[eg@eg-HP-Pavilion-x360-Convertible-14m-cd0xxx ~]$ sudo tail -f /var/log/suricata/fast.log
[sudo] password for eg:
02/28/2026-14:08:29.925863  [**] [1:2027397:1] ET INFO Spotify P2P Client [**] [Classification: Not Suspicious Traffic] [Priority: 3] {UDP} 192.168.1.164:57621 -> 192.1
68.1.255:57621
02/28/2026-14:09:21.304459  [**] [1:2027695:5] ET INFO Observed Cloudflare DNS over HTTPS Domain (cloudflare-dns .com in TLS SNI) [**] [Classification: Misc activity] [
Priority: 3] {TCP} 192.168.1.198:37304 -> 1.1.1.1:443
02/28/2026-14:09:49.267398  [**] [1:2400039:4630] ET DROP Spamhaus DROP Listed Traffic Inbound group 40 [**] [Classification: Misc Attack] [Priority: 2] {TCP} 193.32.16
2.82:37896 -> 192.168.1.198:2222
02/28/2026-14:09:50.381865  [**] [1:2038967:1] ET INFO SSH-2.0-Go version string Observed in Network Traffic [**] [Classification: Misc activity] [Priority: 3] {TCP} 19
3.32.162.82:37896 -> 192.168.1.198:2222
02/28/2026-14:13:30.020122  [**] [1:2027397:1] ET INFO Spotify P2P Client [**] [Classification: Not Suspicious Traffic] [Priority: 3] {UDP} 192.168.1.164:57621 -> 192.1
68.1.255:57621
02/28/2026-14:18:30.070878  [**] [1:2027397:1] ET INFO Spotify P2P Client [**] [Classification: Not Suspicious Traffic] [Priority: 3] {UDP} 192.168.1.164:57621 -> 192.1
68.1.255:57621
02/28/2026-14:19:59.304060  [**] [1:2027695:5] ET INFO Observed Cloudflare DNS over HTTPS Domain (cloudflare-dns .com in TLS SNI) [**] [Classification: Misc activity] [
Priority: 3] {TCP} 192.168.1.198:38022 -> 1.1.1.1:443
02/28/2026-14:20:04.973962  [**] [1:2038967:1] ET INFO SSH-2.0-Go version string Observed in Network Traffic [**] [Classification: Misc activity] [Priority: 3] {TCP} 18
5.247.137.31:54677 -> 192.168.1.198:2222
02/28/2026-14:23:30.118943  [**] [1:2027397:1] ET INFO Spotify P2P Client [**] [Classification: Not Suspicious Traffic] [Priority: 3] {UDP} 192.168.1.164:57621 -> 192.1
68.1.255:57621
02/28/2026-14:24:47.282547  [**] [1:2038967:1] ET INFO SSH-2.0-Go version string Observed in Network Traffic [**] [Classification: Misc activity] [Priority: 3] {TCP} 19
3.32.162.82:45936 -> 192.168.1.198:2222
02/28/2026-14:26:23.418457  [**] [1:2038967:1] ET INFO SSH-2.0-Go version string Observed in Network Traffic [**] [Classification: Misc activity] [Priority: 3] {TCP} 45
.148.10.121:49032 -> 192.168.1.198:2222
02/28/2026-14:28:27.356167  [**] [1:2221050:1] SURICATA HTTP too many warnings [**] [Classification: Generic Protocol Command Decode] [Priority: 3] {TCP} 192.168.1.164:58446 -> 1
92.168.1.244:8000
02/28/2026-14:28:33.397833  [**] [1:2221050:1] SURICATA HTTP too many warnings [**] [Classification: Generic Protocol Command Decode] [Priority: 3] {TCP} 192.168.1.244:8000 -> 19
2.168.1.164:58446
02/28/2026-14:28:35.326333  [**] [1:2027397:1] ET INFO Spotify P2P Client [**] [Classification: Not Suspicious Traffic] [Priority: 3] {UDP} 192.168.1.164:57621 -> 192.168.1.255:5
7621
02/28/2026-14:29:24.183959  [**] [1:2024364:5] ET SCAN Possible Nmap User-Agent Observed [**] [Classification: Web Application Attack] [Priority: 1] {TCP} 192.168.1.198:57100 ->
192.168.1.244:8000
02/28/2026-14:29:24.186393  [**] [1:2024364:5] ET SCAN Possible Nmap User-Agent Observed [**] [Classification: Web Application Attack] [Priority: 1] {TCP} 192.168.1.198:45994 ->
192.168.1.244:8088
02/28/2026-14:29:24.186078  [**] [1:2024364:5] ET SCAN Possible Nmap User-Agent Observed [**] [Classification: Web Application Attack] [Priority: 1] {TCP} 192.168.1.198:57122 ->
192.168.1.244:8000
02/28/2026-14:29:24.186385  [**] [1:2024364:5] ET SCAN Possible Nmap User-Agent Observed [**] [Classification: Web Application Attack] [Priority: 1] {TCP} 192.168.1.198:46008 ->
192.168.1.244:8088
02/28/2026-14:29:24.209509  [**] [1:2024364:5] ET SCAN Possible Nmap User-Agent Observed [**] [Classification: Web Application Attack] [Priority: 1] {TCP} 192.168.1.198:46034 ->
192.168.1.244:8088
02/28/2026-14:29:24.209075  [**] [1:2024364:5] ET SCAN Possible Nmap User-Agent Observed [**] [Classification: Web Application Attack] [Priority: 1] {TCP} 192.168.1.198:57136 ->
192.168.1.244:8000
02/28/2026-14:29:24.205100  [**] [1:2024364:5] ET SCAN Possible Nmap User-Agent Observed [**] [Classification: Web Application Attack] [Priority: 1] {TCP} 192.168.1.198:57124 ->
192.168.1.244:8000
02/28/2026-14:29:24.209498  [**] [1:2024364:5] ET SCAN Possible Nmap User-Agent Observed [**] [Classification: Web Application Attack] [Priority: 1] {TCP} 192.168.1.198:49928 ->
192.168.1.244:443
```

Running the nmap scan from 192.168.1.198 against 192.168.1.244 produces a visible burst in the fast.log. The key signature is ET SCAN Possible Nmap User-Agent Observed, Suricata identifying nmap by name, firing at Priority 1 and classifying it as a Web Application Attack. The scan is visible as a clear transition in the log: background noise, then SURICATA HTTP too many warnings at 14:28 as the version scan hits the web ports, then the nmap burst at 14:29.

SIGNATURE	WHAT IT MEANS
ET SCAN Possible Nmap User-Agent Observed Priority 1 Web Application Attack	Suricata identifies nmap by name. nmap's version detection sends HTTP requests to open web ports with a recognizable user-agent string. Fires from 192.168.1.198 to 192.168.1.244:8000 and :8088, the Splunk ports. Priority 1 is Suricata's highest severity level; this would be a P1 ticket in a production SOC.
SURICATA HTTP too many warnings Priority 3	Fires at 14:28, just before the nmap burst. nmap's version scan sends a high volume of HTTP requests in rapid succession. Suricata flags the session as abnormal due to the rate and pattern of requests, a pre-signature anomaly detection firing ahead of the tool-specific rule.

Section 2: Wazuh HIDS Log Analysis

Wazuh monitors host-level activity on the Ubuntu server by ingesting system logs through a registered agent. The agent ships events to the Wazuh manager running on the AlmaLinux host, where the analysisd engine applies rule correlation across the incoming log stream. Unlike Suricata which watches network packets, Wazuh operates at the operating system layer, reading authentication logs, PAM events, and syslog output directly from the host.

2a. Agent Deployment and Pipeline

The Wazuh agent was deployed on the Ubuntu server and registered to the AlmaLinux manager at 192.168.1.244 over port 1515. The agent ingests SSH authentication events through journald, which captures all sshd output in real time. Once registered, events from the Ubuntu host appear in the manager's alert stream tagged with the agent name pc-s, allowing per-host filtering across the unified alert log.

The Ubuntu server is publicly exposed with SSH open to the internet. This means the Wazuh agent is monitoring authentic inbound attack traffic rather than simulated activity, which produces a continuous and realistic alert stream without any manual triggering.

2b. Brute Force Detection

To view the live alert stream filtered to SSH authentication events I used the command:

sudo tail -f /var/ossec/logs/alerts/alerts.log

```
[eg@eg-HP-Pavilion-x360-Convertible-14m-cd0xxx ~]$ sudo tail -f /var/ossec/logs/alerts/alerts.log
[sudo] password for eg:
tty: ssh

** Alert 1772500814.5224004: - syslog,sshd,authentication_failed,gdpr_IV_35.7.d,gdpr_IV_32.2,gpg13_7.1,hipaa_164.312.b,nist_800_53_AU.14,nist_800_53_AC.7,pci_dss_10.2.4
,pci_dss_10.2.5,tsc_CC6.1,tsc_CC6.8,tsc_CC7.2,tsc_CC7.3,
2026 Mar 02 20:20:14 (pc-s) any->journald
Rule: 5760 (level 5) -> 'sshd: authentication failed.'
Src IP: 47.253.183.81
Src Port: 35852
User: root
Mar 03 01:20:13 pc-s sshd-session[781795]: Failed password for root from 47.253.183.81 port 35852 ssh2

** Alert 1772500836.5224479: - pam,syslog,authentication_success,pci_dss_10.2.5,gpg13_7.8,gpg13_7.9,gdpr_IV_32.2,hipaa_164.312.b,nist_800_53_AU.14,nist_800_53_AC.7,tsc_
CC6.8,tsc_CC7.2,tsc_CC7.3,
2026 Mar 02 20:20:36 eg-HP-Pavilion-x360-Convertible-14m-cd0xxx->/var/log/secure
Rule: 5501 (level 3) -> 'PAM: Login session opened.'
User: root
Mar  2 20:20:35 eg-HP-Pavilion-x360-Convertible-14m-cd0xxx sudo[634971]: pam_unix(sudo:session): session opened for user root(uid=0) by eg(uid=1000)
uid: 1000

** Alert 1772500836.5224979: - syslog,sudo,pci_dss_10.2.5,pci_dss_10.2.2,gpg13_7.6,gpg13_7.8,gpg13_7.13,gdpr_IV_32.2,hipaa_164.312.b,nist_800_53_AU.14,nist_800_53_AC.7,
nist_800_53_AC.6,tsc_CC6.8,tsc_CC7.2,tsc_CC7.3,
2026 Mar 02 20:20:36 eg-HP-Pavilion-x360-Convertible-14m-cd0xxx->/var/log/secure
Rule: 5402 (level 3) -> 'Successful sudo to ROOT executed.'
User: root
Mar  2 20:20:35 eg-HP-Pavilion-x360-Convertible-14m-cd0xxx sudo[634971]:      eg : TTY=pts/2 ; PWD=/home/eg ; USER=root ; COMMAND=/bin/tail -f /var/ossec/logs/alerts/al
erts.log
tty: pts/2
pwd: /home/eg
command: /bin/tail -f /var/ossec/logs/alerts/alerts.log
```

The alert stream showed continuous inbound SSH authentication failures from multiple external source IPs. The following signature categories were observed:

Rule	Level	Description
5760	5	sshd: authentication failed
5710	5	sshd: Attempt to login using a non-existent user
5712	10	sshd: brute force trying to get access to the system

Rule 5712 is Wazuh's brute force correlation rule. It fires when the frequency engine observes repeated rule 5710 events from the same source IP within a correlation window, escalating individual login failures into a confirmed brute force classification. It fired 39 times across the monitoring window against multiple external threat actors.

The most clearly documented escalation came from 8.220.192.64, which attempted sequential logins using non-existent usernames including frappe, huawei, ubuntu, and vyos across a two minute window. Wazuh correlated the individual failures and escalated to rule 5712 at level 10, classifying the activity as a brute force attempt against the host.

Source IP	Behavior	Rule Fired	Level
8.220.192.64	Sequential invalid user attempts: frappe, huawei, ubuntu, vyos	5712	10
144.31.11.68	Repeated root and invalid user attempts across extended session	5710	5
185.252.232.247	Sustained root login attempts, 370 log entries across monitoring window	5710	5
91.202.233.33	Rapid sequential attempts with connection resets after each failure	5710	5

2c. Detection Notes

The credential stuffing usernames observed across the attack traffic included root, admin, kali, postgres, test, frontend, openhabian, sshadmin, frappe, huawei, ubuntu, and vyos. The variety of usernames indicates automated tooling cycling through a wordlist rather than targeted credential guessing. This is characteristic of internet background noise on any publicly exposed SSH service.

Rule 5720, Wazuh's highest severity brute force escalation, requires 8 authentication failures from the same source IP within its correlation window. On this host, Fail2ban bans source IPs after 10 failures and the ban window resets the per-source failure count before the 5720 threshold is reached. This is an interaction between two defensive layers: Fail2ban blocks the attacker before Wazuh can escalate to the highest severity classification. In a production environment these thresholds would be tuned together so detection fires before automated blocking interrupts the correlation chain. The 5712 escalations captured here represent the highest severity events that the combined Fail2ban and Wazuh configuration allows to complete.

2d. What This Shows

The Wazuh section demonstrates host-layer log ingestion and rule correlation operating against real external attack traffic. The agent pipeline from Ubuntu journald to the AlmaLinux manager is functional end to end. The analysisd engine is correctly correlating individual authentication failures into brute force classifications without manual tuning. The interaction between Fail2ban and Wazuh illustrates a real operational consideration: defensive tools deployed on the same host can interfere with each other's detection thresholds, and understanding that interaction is part of building a coherent security stack rather than just deploying individual tools in isolation.

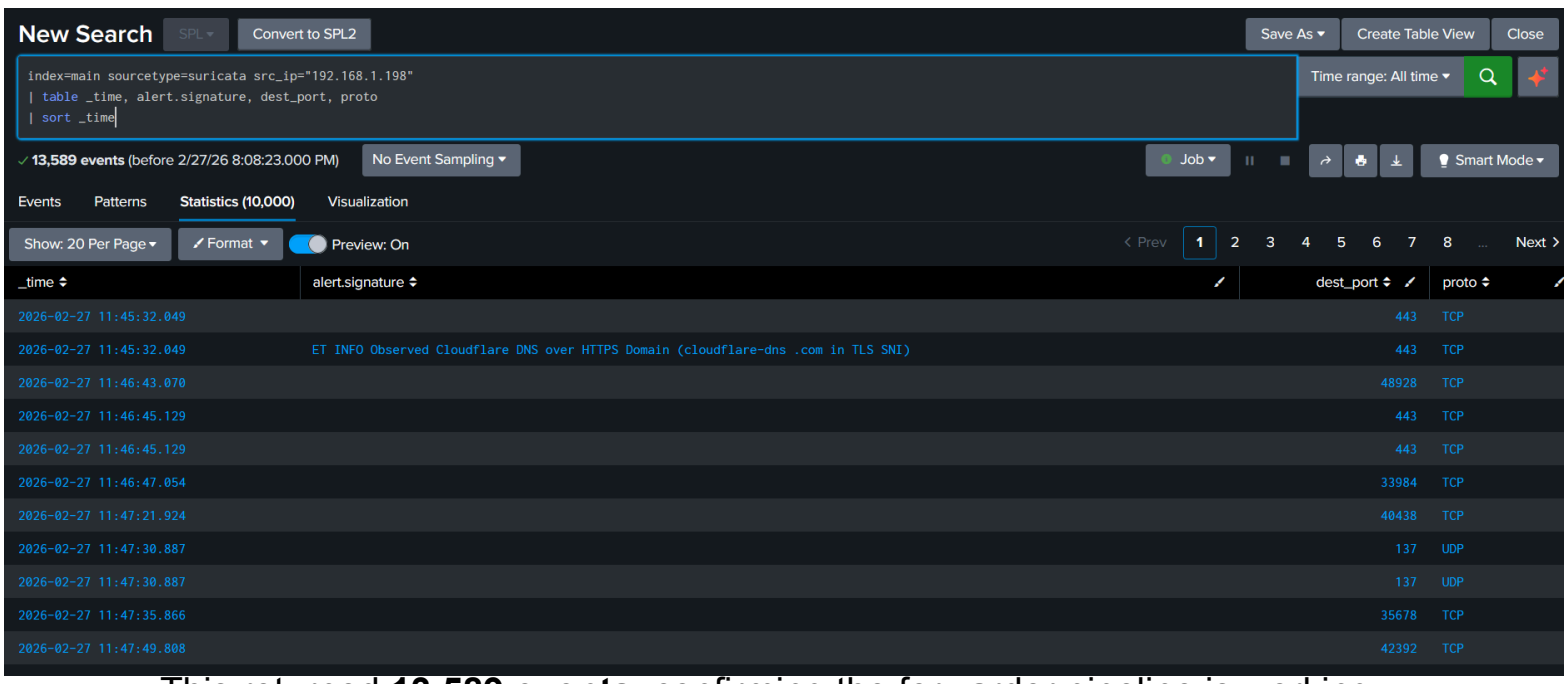
Section 3: Splunk SIEM Analysis -

Suricata writes alerts to eve.json in structured JSON format. Splunk's Universal Forwarder monitors that file and indexes each event under sourcetype=suricata. From there, any alert Suricata fires is queryable in the SIEM using SPL (Search Processing Language).

Query 1 - All events from the scan host

The first thing to do with any investigation in Splunk is confirm data is present and filter to the source you care about. This query pulls every Suricata event from the scan host and returns a table of timestamp, signature, destination port, and protocol:

```
index=main sourcetype=suricata src_ip="192.168.1.198"
| table _time, alert.signature, dest_port, proto
| sort _time
```

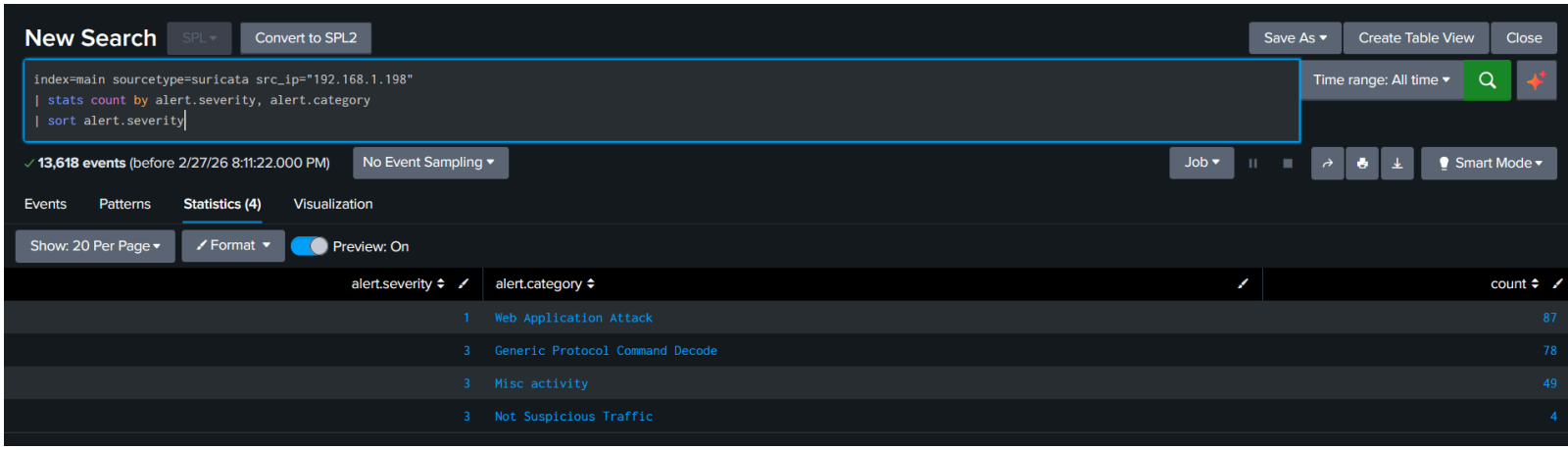


This returned **13,589 events**, confirming the forwarder pipeline is working end to end as Suricata detected it, EVE JSON captured it, the forwarder shipped it, and Splunk indexed it.

Query 2 - Alert severity and category breakdown

Once data is confirmed, the next step is categorization and understanding what types of alerts fired and how severe they were. This is the triage query:

```
index=main sourcetype=suricata src_ip="192.168.1.198"
| stats count by alert.severity, alert.category
| sort alert.severity
```



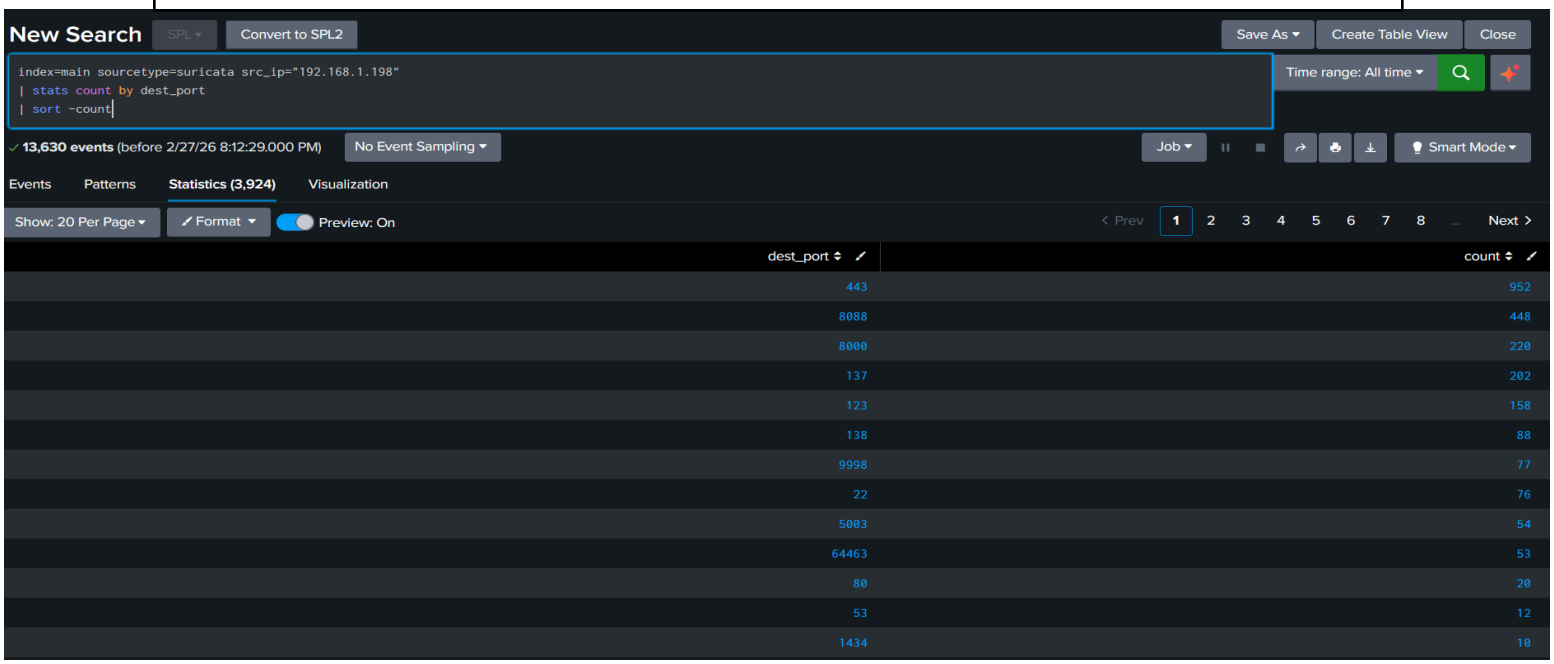
The results broke down across four categories across 13,618 events:

Severity	Category	Count	What it means
1	Web Application Attack	87	Highest priority, nmap's NSE scripts probing web ports triggered these. Would be a P1 ticket in a production SOC.
3	Generic Protocol Command Decode	78	SSH and HTTPS protocol anomalies, the SSH invalid banner signatures land here.
3	Misc Activity	48	Mix of scan-related and background traffic, Cloudflare DNS observations, ET INFO signatures.
3	Not Suspicious Traffic	4	Legitimate traffic like Spotify P2P. Suricata logs everything, not every alert is a threat.

Query 3 - Top destination ports

Port analysis shows what the scan was most interested in, which services it spent the most time probing:

```
index=main sourcetype=suricata src_ip="192.168.1.198"
| stats count by dest_port
| sort -count
```



dest_port	count
443	952
8088	448
8000	220
137	202
123	158
138	88
9998	77
22	76
5003	54
64463	53
80	20
53	12
1434	10

Port 443 led with 952 events, the Wazuh HTTPS management interface. Port 8088 (448 events) and 8000 (220 events) followed, both Splunk ports. The scan was hitting management interfaces the hardest, which makes sense: those are the highest-value services on the host.

Query 4 - Event table with timestamps

Reformatting Splunk's internal epoch time into a readable timestamp makes it easier to build an event timeline and confirm the order of activity during the investigation window:

```
index=main sourcetype=suricata src_ip="192.168.1.198"
| eval time=strftime(_time,"%D %R:%S")
| table time, alert.signature, alert.severity, dest_port
| sort _time
```

New Search

```
index=main sourcetype=suricata src_ip="192.168.1.198"
| eval time=strftime(_time,"%D %R:%S")
| table time, alert.signature, alert.severity, dest_port
| sort _time
```

26,233 events (before 2/28/26 4:15:54.000 PM)

time	alert.signature	alert.severity	dest_port
02/28/26 11:00:19	ET INFO Observed Cloudflare DNS over HTTPS Domain (cloudflare-dns .com in TLS SNI)	3	443
02/28/26 10:50:08	ET INFO Observed Cloudflare DNS over HTTPS Domain (cloudflare-dns .com in TLS SNI)	3	443
02/28/26 10:39:40	ET INFO Observed Cloudflare DNS over HTTPS Domain (cloudflare-dns .com in TLS SNI)	3	443
02/28/26 10:29:27	ET INFO Observed Cloudflare DNS over HTTPS Domain (cloudflare-dns .com in TLS SNI)	3	443
02/28/26 10:20:10	SURICATA SSH invalid banner	3	44242
02/28/26 10:19:42	SURICATA SSH invalid banner	3	39520
02/28/26 10:19:21	SURICATA SSH invalid banner	3	35136
02/28/26 10:17:51	ET INFO Observed Cloudflare DNS over HTTPS Domain (cloudflare-dns .com in TLS SNI)	3	443
02/28/26 10:17:47	SURICATA SSH invalid banner	3	58222
02/28/26 10:17:47	SURICATA Applayer Detect protocol only one direction	3	58222

This query returned 26,233 events, higher than the 13,589 returned in Query 1. The difference reflects continued background traffic accumulation from 192.168.1.198 between the original scan session and the time this query was run, which confirms the monitoring pipeline is persistent and live. The table makes individual events readable at a glance: Cloudflare DNS observations hitting port 443, SURICATA SSH invalid banner signatures on high ephemeral ports, and SURICATA Applayer detections visible in sequence. This view is useful for building an exact timeline of activity when the aggregate queries have already established what fired and how often.

What this shows:

The four queries above demonstrate a complete log aggregation and analysis workflow across two data sources. Suricata captures activity at the packet level and writes structured EVE JSON, giving you the raw signal. Splunk ingests that output and makes it measurable: event counts by source, severity distribution across alert categories, port-level targeting patterns, and a timestamped event timeline. Neither tool alone answers the full question. Suricata tells you what happened; Splunk tells you how much, how often, and in what order. Correlating Wazuh host-layer data against Suricata network-layer alerts in the same SIEM adds a second confirmation axis, when two independent detection engines flag the same activity, confidence in the finding increases. That cross-source analysis is the core skill this section demonstrates.

Section 4: Findings and Conclusions -

4a. Key Findings

Finding	Detail	Severity
Reconnaissance detected by signature	Suricata fired ET SCAN Possible Nmap User-Agent Observed at Priority 1, classifying the activity as a Web Application Attack. The signature matched nmap's version detection HTTP requests to ports 8000 and 8088. Suricata identified the tool by name, not just by behavior, demonstrating that the ET ruleset covers known scan tooling.	HIGH
Pre-signature anomaly preceded tool-specific rule	SURICATA HTTP too many warnings fired at 14:28, approximately one minute before the ET SCAN signature at 14:29. The anomaly detection engine flagged abnormal HTTP request volume before the tool-specific rule matched, providing an additional detection layer independent of signature coverage.	MEDIUM
Management interfaces were the primary scan target	Port analysis showed port 443 (952 events), 8088 (448 events), and 8000 (220 events) as the top three targeted ports. These correspond to the Wazuh HTTPS management interface and the two Splunk service ports. High-value administrative interfaces received the most probe traffic.	HIGH
Inbound threat intel matches on scan host IP	grep output filtered to 192.168.1.198 showed inbound Spamhaus DROP, ET CINS, and ET COMPROMISED hits arriving at the host from external IPs. These are unrelated to the scan activity and represent internet background noise from automated bots targeting the exposed SSH port.	INFO
Wazuh brute force detection confirmed against live inbound attack traffic	SSH authentication failures from multiple external IPs triggered Wazuh rule 5712 (level 10) 39 times across the monitoring window, automatically classifying repeated invalid login attempts as confirmed brute force activity. Rule 5720 did not fire because Fail2ban bans source IPs before Wazuh's higher frequency threshold is reached.	HIGH
Full pipeline integrity confirmed end to end	Suricata detected activity at the packet level, wrote structured EVE JSON, the Universal Forwarder ingested and shipped the file, and Splunk indexed 13,589 events scoped to the scan session. The pipeline from sensor to SIEM operated without gaps across the monitoring window.	INFO

4b. Analyst Notes

The most significant observation across both tools is the consistency of the detection. Suricata and Wazuh flagged the same activity independently, from the network layer and the host layer respectively, without any coordination between them. When two separate detection engines converge on the same event, confidence in the finding increases significantly. In a production environment this would support escalation to a confirmed incident rather than a low-confidence alert.

The inbound threat intelligence hits on 192.168.1.198 are worth noting separately. The Spamhaus, CINS, and ET COMPROMISED signatures firing against the scan host's SSH port have nothing to do with the nmap activity. They are automated bots continuously probing internet-exposed SSH services. Distinguishing these from the intentional scan traffic is a core triage skill: both appear in the same log, both fire alerts, but they represent entirely different threat categories that require different responses.

Noise reduction is not a configuration problem, it is an analyst judgment problem. Suricata logging Spotify P2P and Cloudflare DNS is correct behavior. The analyst's responsibility is knowing which signatures represent background activity and which represent actionable events. That distinction cannot be automated away.

4c. Conclusions

This monitoring setup demonstrates functional full-stack visibility across the network and host layers. Suricata provides real-time network detection with structured output suitable for SIEM ingestion. Wazuh provides host-layer correlation and automated rule escalation. Splunk provides the investigation layer where both data sources can be queried, measured, and compared against each other.

The investigation workflow documented in this report mirrors standard SOC triage practice: confirm data is present, categorize what fired, identify the highest severity events, drill into specific signatures, and build a timeline.

Each step builds on the previous one. Starting with a broad count and narrowing to specific events is not just a Splunk workflow, it is how alert triage works regardless of the tooling.

4d. What I Would Do Next

This setup is functional and the detection pipeline works end to end, but there are gaps I have already identified and am actively working to address.

The most immediate gap is alert suppression. Known good signatures like Spotify P2P and Cloudflare DNS fire continuously and add noise. The fix is straightforward, build a Suricata threshold configuration to suppress repetitive low value signatures without disabling the rules entirely. The goal is not to stop logging them, it is to stop them from competing with actionable alerts during triage.

The second gap is automated response. Wazuh correctly escalates SSH brute force attempts to High severity through its rule chain, but detection stops there. The next step is configuring Wazuh active response to trigger a Firewalld block on confirmed brute force source IPs automatically. Detection without response is only half the loop.

Log retention policy is also undefined. The current setup does not specify how long Splunk indexes events before rolling over. In a production environment this would be a compliance and forensics concern. Formalizing a retention window is a small configuration change with significant operational implications.

Once those gaps are closed, the next phase is to start expanding visibility. Planned additions include custom Splunk dashboards for alert frequency trending over time, Windows event log ingestion from the Windows 10 Desktop via a Wazuh agent, and network flow analysis to complement the signature based detection already in place. The objective is building a monitoring environment that more closely mirrors what an enterprise SOC operates against, not just a detection testbed but a network with layered controls, defined response procedures, and documented coverage gaps.